

sedfit

User Manual

Photometry in, redshifts out

Version 0.1.0

Contents

1	What this does	4
2	Installing	4
2.1	Get Python	4
2.2	Get the code	5
2.3	Create an environment and install	5
2.4	What “editable” means	6
2.5	Optional extras	6
2.6	Choosing a stellar library: MILES or C3K	7
2.7	Verify the install	8
3	Before you start	8
3.1	The directory layout it expects	8
3.2	What a photometry file must contain	9
3.3	The three files you write	9
4	The verbs	11
5	Your first run	12
6	Common tasks	13
6.1	A note on long commands	13
6.2	Add galaxies to the sample	13
6.3	Fit one galaxy	13
6.4	Check a command before running it	13
6.5	Build tables without fitting	13
6.6	Run only some galaxies	13
6.7	Run only one recipe	14
6.8	Correct for Milky Way dust	14
6.9	Redo a fit that already exists	14
6.10	Use more or fewer processors	14
6.11	Redraw figures	14
6.12	See everything that has been run	15
7	Every flag, explained	15
7.1	Shared by <code>fit</code> , <code>batch</code> and <code>run</code>	15
7.2	<code>roster</code>	15
7.3	<code>build</code>	15
7.4	<code>fit</code>	16
7.5	<code>batch</code> and <code>run</code>	16
8	What comes out	17
8.1	The directory tree	17
8.2	What each file is	18
8.3	The numbers you probably want	18
9	Reading the figures	18
10	When something goes wrong	19

11 Getting more out of it	20
11.1 Several recipes at once	20
11.2 Comparing template sets	21
11.3 The central log	21
11.4 A worked example to copy	21
11.5 Where to read next	21

1 What this does

sedfit takes photometry you already have for a set of galaxies and fits each one's spectral energy distribution to estimate its redshift.

You give it three things:

1. a **sample catalog** — a spreadsheet listing your galaxies, one row each;
2. a **campaign config** — a settings file saying which photometry files to expect and how to combine them;
3. a **fit config** — a settings file saying how to fit.

It then does three things:

1. **Builds** one combined photometry table per galaxy, putting every instrument on a common flux scale.
2. **Fits** that table with one of two engines and estimates a redshift with uncertainties.
3. **Records** everything — every input file is hashed, every setting is saved next to the results, and every run is logged in one central file.

Fluxes are in microjansky on the AB system throughout. Everything for one galaxy lands under that galaxy's own directory.

2 Installing

These instructions assume the machine has nothing set up yet. Commands are the same on all three operating systems unless stated otherwise.

2.1 Get Python

sedfit needs **Python 3.11 or newer** plus a scientific stack (NumPy, SciPy, Astropy, Pandas, Matplotlib).

Use **Miniconda**. Conda installs Python and the scientific libraries as pre-built binaries, which avoids the compiler toolchain some of these packages otherwise need, and it keeps each project's libraries separate so one project cannot break another.

Windows

1. Go to <https://www.anaconda.com/download/success> and download the **Miniconda Windows 64-bit installer (.exe)**.
2. Run the installer and accept the defaults.
3. From the Start menu, open **Anaconda Prompt**. Every command in this manual is typed there.

macOS

1. Download the **Miniconda macOS installer** for your chip (Apple silicon = **arm64**, Intel = **x86_64**).
2. Run it, then open **Terminal**.

Linux

1. Download the Miniconda Linux **x86_64** installer script.
2. Run **bash Miniconda3-latest-Linux-x86_64.sh**, then open a new terminal.

Check that conda is available:

```
conda --version
```

You should see something like `conda 24.9.2`.

2.2 Get the code

The source lives at <https://github.com/chrishopp12/sed-fitting>.

With git. On Windows, install Git for Windows from <https://git-scm.com/download/win> first; macOS and Linux usually have git already. Then:

```
git clone https://github.com/chrishopp12/sed-fitting.git
```

This creates a folder named `sed-fitting`. Move into it:

```
cd sed-fitting
```

Without git. Open the page in a browser, click the green **Code** button, choose **Download ZIP**, and unzip it. You get a folder named `sed-fitting-main`. Move into it:

```
cd sed-fitting-main
```

You are in the right folder if it contains a file named `pyproject.toml`.

The download is about 20 MB and unpacks to about 70 MB

Most of that is the packaged template libraries — the galaxy spectra the fit compares your photometry against. They ship with the code so a fit is reproducible without downloading anything else.

2.3 Create an environment and install

Conda route (recommended, all platforms).

```
conda create -n sedfit python=3.11
```

Answer `y` when it asks. Then activate it:

```
conda activate sedfit
```

Your prompt now starts with `(sedfit)`. Install the package:

```
pip install -e .
```

That reads `pyproject.toml` and pulls in every dependency.

If a dependency fails to build, install the scientific stack from conda-forge first and then install `sedfit` without re-resolving:

```
conda install -c conda-forge numpy scipy pandas astropy matplotlib
```

```
pip install -e . --no-deps
```

pip / venv route (no conda). Only use this if Python 3.11+ is already installed and on your PATH.

Windows:

```
python -m venv .venv
```

```
.venv\Scripts\activate
```

macOS / Linux:

```
python3 -m venv .venv
```

```
source .venv/bin/activate
```

Then, on any platform:

```
pip install -e .
```

2.4 What “editable” means

`pip install -e .` installs the package **in place**: Python runs the source files in the folder you downloaded, rather than a copy hidden in the environment. If you `git pull` a newer version, the change takes effect immediately with no reinstall.

2.5 Optional extras

The base install runs the assembly path and the quick engine — enough for your first run. Four extras exist:

```
pip install -e ".[easy]"      # the official easy-py engine
```

```
pip install -e ".[prospector]"  # Prospector + FSPS
```

```
pip install -e ".[test]"       # pytest, for the check below
```

```
pip install -e ".[dered]"      # astroquery, for --deredden
```

The last two are easy to overlook: `pytest` is not in the base install, so `python -m pytest` below fails without `".[test]"`, and the automatic E(B–V) lookup that `--deredden` performs needs `astroquery`. You can install several at once: `pip install -e ".[easy,test,dered]"`.

Prospector on Windows

Prospector needs FSPS, which compiles from Fortran source and is difficult to build on Windows. The base install and both easy engines run on any platform; if you need Prospector, use Linux or macOS.

Prospector also needs a copy of the FSPS data files and an environment variable pointing at them, set before every run:

```
git clone https://github.com/cconroy20/fsps ~/fsps
```

```
export SPS_HOME=~/fsps
```

2.6 Choosing a stellar library: MILES or C3K

Prospector builds a galaxy's light out of model stellar spectra, and FSPS ships two libraries to draw them from: **MILES** (measured from real stars) and **C3K** (computed from stellar atmosphere models). They do not give the same answer, so every Prospector config has to say which one it used — that is the `stellar_library` field, either "miles" or "c3k".

The awkward part is that the library is fixed when FSPS is **compiled**, not when it runs. No setting switches between them. You need a separate environment for each, and plain `pip install fsps` gives you the MILES build.

MILES works out of the box:

```
conda env create -f envs/sedfit_miles.yml
conda activate sedfit_miles
pip install -e .
```

The .yaml files do not install sedfit itself

They build the environment around it — Python, the scientific stack, Prospector, FSPS — but not the package. Creating an environment and stopping there gets you `sedfit: command not found`. Always follow `conda env create` with `conda activate` and `pip install -e .` from inside the repository folder.

C3K takes one extra step. Creating the environment is not enough: its .yaml installs the same pre-built package everyone else gets, which is MILES. You have to rebuild FSPS afterwards, and that compiles from Fortran source, so the environment needs a compiler first.

```
conda env create -f envs/sedfit_c3k.yml
conda activate sedfit_c3k
conda install -c conda-forge compilers
FFLAGS="-DMILES=0 -DC3K=1" pip install --force-reinstall \
    --no-cache-dir --no-binary fsps fsps==0.4.7
pip install -e .
```

That takes a few minutes. macOS ships no Fortran compiler, which is why the `compilers` line comes first.

Check what a given environment actually has:

```
python -c "import fsps; \
    print(fsps.StellarPopulation(zcontinuous=1).libraries)"
```

The middle entry is the spectral library:

```
sedfit_miles -> (b'mist', b'miles', b'DL07')
sedfit_c3k   -> (b'mist', b'c3k_a', b'DL07')
```

If the C3K environment still reports `miles`, the rebuild did not take. Run it again and watch for compiler errors — a failed build leaves the previous install in place rather than removing it. If it will not go through at all and you have another environment that already reports `c3k_a`, clone that instead and install into the clone:

```
conda create -n sedfit_c3k --clone <the working env>
conda activate sedfit_c3k
pip install -e .
```

The fit checks this for you

Every Prospector fit compares the config's `stellar_library` against the library the live FSPS build reports, before it does any work, and stops if they disagree:

```
stellar_library mismatch:  the config declares 'c3k' (FSPS stamp
'c3k_a') but the live build reports 'miles'; run in the matching
sedfit env
```

Activating the wrong environment therefore costs you a clear error, not a quietly wrong answer. Nothing is written when this fires, so there is no half-finished run to clean up.

Results from the two libraries are not interchangeable. Compare fits within one library, or treat the difference between them as one of the things you are measuring.

2.7 Verify the install

```
sedfit --help
```

You should see a list of seven verbs: `roster`, `build`, `fit`, `batch`, `run`, `plot`, `manifest`. Then run the test suite (this needs the `test` extra above, which installs `pytest`):

```
python -m pytest tests/
```

Everything should pass. Tests for an engine you did not install skip themselves rather than failing, so a base install reports a couple of skips and no failures.

3 Before you start

`sedfit` does not download photometry. It expects the photometry files to be on disk already, laid out one directory per galaxy.

3.1 The directory layout it expects

```
data_root/
  gal_0001/
    Photometry/
      gal_0001_catalog.csv
      gal_0001_measured.csv
    SPHEREx/
      table_photometry.csv
  gal_0002/
    Photometry/
      ...
```

You choose where `data_root` is and what the galaxy directories are called; the campaign config points at them. Names like `Photometry/` are conventions the campaign config declares,

not rules baked into the code.

3.2 What a photometry file must contain

Every source CSV needs four columns:

Column	Meaning
<code>band</code>	the filter name, e.g. <code>Legacy_g</code>
<code>flux_uJy</code>	flux in microjansky
<code>flux_err_uJy</code>	its uncertainty
<code>source</code>	where the measurement came from

The `source` column matters more than it looks. One file can hold measurements from several archives, and `sedfit` tells them apart by the start of this string — `Legacy_DR9`, `SDSS_DR17_cModel`, and so on. It is how the same band can come from two different places in one file.

Three more columns are used if you supply them, and ignored if you do not: `target_ra` and `target_dec`, which are checked against the galaxy’s catalog position and must agree within 0.5 arcsec; and `flags`, which becomes the fit’s quality-gate column for aperture measurements. Any other column is left alone.

Where the legal band names live

`sedfit/data/registry.json` lists every band the package knows — 37 of them across CFHT, GALEX, J-PLUS, Legacy, Pan-STARRS, SDSS, SPHEREx and WISE. Open it to check a spelling; an unrecognized name is a hard error naming what it did not recognize. Watch for the ones that do not read the way you would guess: Legacy has `g r i z` and no `u`, and the WISE bands are `WISE_W1–W4` while the *provider* tokens for them are `unwise` and `allwise`. SPHEREx channels are the exception — they are built per galaxy and are not listed in the registry.

3.3 The three files you write

1. The sample catalog (a CSV, one row per galaxy). This is the file you edit as the sample grows.

```
name,ra_deg,dec_deg,dir,z_ref_kind,z_ref
gal_0001,217.51,56.99,gal_0001,spec,0.1043
gal_0002,217.62,57.02,gal_0002,spec,0.1058
gal_0003,217.44,56.95,gal_0003,reference,
```

Column	Meaning
<code>name</code>	the galaxy’s name; must be unique
<code>ra_deg, dec_deg</code>	position in degrees
<code>dir</code>	its directory under <code>data_root</code> (optional; defaults to the name)
<code>z_ref_kind</code>	<code>spec</code> , <code>phot</code> , or <code>reference</code>
<code>z_ref</code>	the known redshift; leave blank for reference
<code>label</code>	the filename stem (optional; defaults from the name)

`z_ref_kind` says where the comparison redshift comes from. Use `spec` or `phot` when the galaxy has its own measured value, and `reference` when it should adopt the sample-wide redshift set in the campaign config. Columns `sedfit` does not recognize are ignored and reported, so you can keep your own notes in the same file.

2. The campaign config (`campaign.json`). Everything shared across galaxies. A minimal one:

```
{
  "schema_version": 1,
  "sample": "my_sample",
  "data_root": "data",
  "reference_redshift": 0.1050,
  "position_authority": "Legacy DR9 positions",
  "sources": {
    "legacy": {
      "path": "Photometry/{label}_catalog.csv",
      "bands": ["Legacy_g", "Legacy_r", "Legacy_z"],
      "kind": "catalog",
      "provider": "legacy"
    }
  },
  "recipes": {
    "plain": {
      "reference": "none",
      "sources": [{"source": "legacy", "role": "float"}],
      "spherex": null
    }
  }
}
```

`data_root` is relative to the campaign file — so this campaign sits next to a `data/` folder holding the galaxy directories. A *source* is one photometry file and the bands you expect from it; `{label}` is replaced with each galaxy's filename stem. `provider` must be one of `legacy`, `unwise`, `allwise`, `galax`, `sdss`, `panstarrs`, `jplus`, `hst`, `aperture`, `sersic` — each maps to the prefix its rows must carry in the `source` column (`legacy` → `Legacy_`, `unwise` → `unWISE_Legacy_`, and so on). A *recipe* is one way to combine those sources into a table — you can define several and run them all. `"spherex": null` in a recipe means *deliberately leave the spectrum out of this one*, which is what you want when the recipe is broadband-only.

reference_redshift is required as soon as you use it

The moment one catalog row says `z_ref_kind: reference`, the campaign must declare `reference_redshift` — that is where the redshift comes from. `sedfit roster` does **not** check this: it writes a clean roster, and the *next* command fails with `z_ref_kind 'reference' requires the roster to declare reference_redshift`. If a verb fails that way right after a successful roster, this is why.

3. The fit config (e.g. `quick.json`). How to fit:

```
{
  "schema_version": 2,
  "backend": "eazy",
  "name": "quick_v1",
  "err_floor": 0.05,
  "min_snr_broadband": 2.0,
  "min_valid_bands": 5,
```

```

    "eazy": {
      "engine": "quick",
      "z_min": 0.0,
      "z_max": 1.0,
      "z_step": 0.001,
      "templates": "brown14_vac_cosmos160",
      "template_pattern": "*.dat"
    }
  }

```

You must name a template set

There is no default. The template set is the single biggest influence on the redshift you get out, so the code makes you choose it deliberately. Name any of the sets shipped in `sedfit/data/templates/` — `brown14_vac_cosmos160` is the recommended one — or give a path to your own directory of spectra.

And set `template_pattern` with it

`template_pattern` says which files in that directory are spectra, and it defaults to `*_spec.dat`. The recommended set mixes two naming conventions: 129 Brown+14 spectra named `*_spec.dat` and 31 COSMOS templates named differently. So the default silently fits **129 of the 160** templates. Set `"template_pattern": "*.dat"` to use the whole basis. The code warns when a pattern selects only part of a set, but that warning is easy to lose in a batch log.

Always set the redshift range

`z_min` and `z_max` default to **0.05** and **0.16** — a narrow window left over from the survey this package was written for. A config that omits them fits that window and says nothing about it, so every galaxy comes back between 0.05 and 0.16 whether or not it belongs there. Set them for your own sample, as the example above does.

4 The verbs

Verb	What it does
<code>roster</code>	Reads your sample catalog and campaign config, checks what is actually on disk, and writes <code>roster.json</code>
<code>build</code>	Combines each galaxy's photometry into one fit-ready table
<code>fit</code>	Fits one galaxy with one recipe
<code>batch</code>	Fits every galaxy, in parallel, and writes a report
<code>run</code>	Does <code>roster</code> , <code>build</code> and <code>batch</code> in one command
<code>plot</code>	Redraws the figures for a run you already have
<code>manifest</code>	Summarizes every run recorded so far

The normal working order is `roster` → `build` → `batch`. The `run` verb chains those three, and is what you want most of the time.

`run` always rebuilds every table and always resumes, and neither is overridable — it has no `--build`, `--no-resume` or `--dry-run`. Reach for `roster` and `batch` separately when you need either off.

A command that fails tells the shell so: **batch** and **run** exit nonzero if any job failed, and **manifest** exits nonzero if anything it checked is missing. That is what a script should test.

Every verb explains itself:

```
sedfit --help      sedfit build --help
```

5 Your first run

Put your sample catalog and campaign config next to each other, then:

Step 1 — check what the code can see. This writes nothing:

```
sedfit roster --catalog sample.csv --campaign campaign.json --dry-run
```

You get a line per galaxy-and-source that did not work out:

```
sample.csv:  3 rows; using name, ra_deg, dec_deg, dir, z_ref_kind, z_ref
3 catalog rows -> 3 targets
    1 dropped, 8 kept
    dropped gal_0003/legacy:  Photometry/gal_0003_catalog.csv:  absent
dry run:  nothing written
```

Read that carefully before going on — it is telling you which files it found and which it did not.

Step 2 — write the roster. Drop `--dry-run`:

```
sedfit roster --catalog sample.csv --campaign campaign.json
```

This writes `roster.json` next to the campaign file, plus `roster.json.report.csv`, which has one row per galaxy-and-source with the reason it was kept, partially kept, or dropped.

Never edit roster.json by hand

It is generated. If something in it is wrong, fix the sample catalog or the campaign config and run **sedfit roster** again — it takes seconds. Hand edits are silently overwritten the next time anyone regenerates it.

Step 3 — build and fit everything:

```
sedfit batch --roster roster.json --config quick.json --build
```

You get a line per job as it finishes:

```
6 jobs (eazy), 3 worker(s); 0 pair(s) not applicable
[1/6] ok          gal_0001/plain z50=0.1043
[2/6] ok          gal_0002/plain z50=0.1061
...
6 ok, 0 failed, 0 skipped -> /path/to/batch_report.csv
```

Or do all three steps at once:

```
sedfit run --catalog sample.csv --campaign campaign.json \
    --config quick.json
```

6 Common tasks

6.1 A note on long commands

A few commands below are too wide for one line and are shown broken across two, with a trailing `\`. That is the continuation character for bash and zsh (macOS, Linux).

Shell	Continuation character
macOS / Linux (bash, zsh)	<code>\</code>
Windows Command Prompt / Anaconda Prompt	<code>^</code>
Windows PowerShell	<code>`</code> (backtick)

On Windows, either swap the trailing `\` for the right character, or — simplest and always correct — delete the line break and paste the whole thing as one long line.

6.2 Add galaxies to the sample

Add rows to your sample catalog, then regenerate and re-run:

```
sedfit roster --catalog sample.csv --campaign campaign.json
sedfit batch --roster roster.json --config quick.json --build
```

Galaxies that were already fitted are skipped automatically, so only the new ones cost time. This is the everyday loop — unless `--build` produces a different table than last time, which changes the run fingerprint and makes it a new fit.

6.3 Fit one galaxy

```
sedfit fit --roster roster.json --target gal_0001 \
  --recipe plain --config quick.json
```

6.4 Check a command before running it

`--dry-run` validates everything and writes nothing. It exists on `roster`, `build` and `fit`; `batch` and `run` have no dry run, so validate one galaxy with `fit --dry-run` before launching a sweep. Use it whenever you have changed a config:

```
sedfit fit --roster roster.json --target gal_0001 \
  --recipe plain --config quick.json --dry-run
```

6.5 Build tables without fitting

```
sedfit build --roster roster.json
```

Add `--target` or `--recipe` to narrow it down.

6.6 Run only some galaxies

Put the names in a CSV with a `name` column — your sample catalog works directly — and pass it:

```
sedfit batch --roster roster.json --config quick.json \  
  --targets subset.csv
```

Or a single one with `--target gal_0001`.

6.7 Run only one recipe

```
sedfit batch --roster roster.json --config quick.json --recipe plain
```

6.8 Correct for Milky Way dust

```
sedfit batch --roster roster.json --config quick.json \  
  --build --deredden
```

This looks up the extinction at each galaxy's position and writes the `_dered` table.

The automatic lookup needs astroquery

It is not in the base install or in either conda environment. Either install it (`pip install -e "[dered]"`), or skip the lookup by giving the value yourself with `--mw-ebv 0.0158`. Without either you get auto E(B-V) lookup needs astroquery (`pip install astroquery`); or pass an explicit value with `--mw-ebv`.

Use --build with --deredden

`--deredden` tells the fit to look for the dereddened table. If you do not also pass `--build`, that table may not exist and the fit will tell you so.

6.9 Redo a fit that already exists

By default a batch skips galaxies already fitted with the same settings. To force it:

```
sedfit batch --roster roster.json --config quick.json --no-resume
```

If it then complains that the existing run came from different software, add `--force`.

6.10 Use more or fewer processors

```
sedfit batch --roster roster.json --config quick.json --workers 4
```

`--workers 1` runs everything in one process, which makes error messages much easier to read. Use it when debugging.

6.11 Redraw figures

```
sedfit plot --run-dir gal_0001/SED/plain/eazy/64f456f8835f-quick_v1
```

6.12 See everything that has been run

```
sedfit manifest --roster roster.json
```

```
792 rows, 792 current runs, 0 superseded
64f456f8835f ok gal_0001/SED/plain/eazy/... z50=0.1043
```

7 Every flag, explained

`sedfit <verb> --help` always prints the current list. This section explains what the less obvious ones mean.

7.1 Shared by fit, batch and run

Flag	Meaning
<code>--force</code>	Replace an existing run even though it was produced by different package versions. Without this, the code refuses, because the old and new results would not be comparable.
<code>--allow-registry-drift</code>	Record a warning instead of stopping when a filter curve has changed since the table was built.
<code>--deredden</code>	Fit the Milky-Way-corrected (<code>_dered</code>) table. On <code>batch</code> <code>--build</code> and on <code>run</code> , it also makes the build itself dereddening, so one flag both produces and fits the corrected tables.
<code>--no-plots</code>	Skip the figures. Faster for large sweeps.

7.2 roster

Flag	Meaning
<code>--catalog</code>	Your sample catalog CSV. Required.
<code>--campaign</code>	Your campaign config JSON. Required.
<code>--out</code>	Where to write the roster. Defaults to <code>roster.json</code> beside the campaign file.
<code>--dry-run</code>	Report what would be declared; write nothing.

7.3 build

Flag	Meaning
<code>--roster</code>	The roster JSON. Required.
<code>--target</code>	Build one galaxy; omit for all of them.
<code>--recipe</code>	Build one recipe; omit for every applicable one.
<code>--dry-run</code>	Validate every pair; write nothing.

Flag	Meaning
<code>--deredden</code>	Write the Milky-Way-corrected <code>_dered</code> table <i>instead of</i> the as-measured one. One build writes one table, so run <code>build</code> twice — once with the flag, once without — if you want both on disk.
<code>--mw-ebv</code>	Use one E(B–V) for every galaxy instead of looking each one up. Needs <code>--deredden</code> .

7.4 fit

Flag	Meaning
<code>--roster</code>	The roster JSON. Required.
<code>--target</code>	Which galaxy.
<code>--recipe</code>	Which recipe's built table to fit.
<code>--config</code>	The fit config JSON.
<code>--label</code>	A suffix for the output directory name. Defaults to the <code>name</code> in the fit config.
<code>--jobs</code>	A JSON list of { <code>target</code> , <code>recipe</code> , <code>config</code> } objects run in order, instead of naming one. Each may also carry <code>label</code> and <code>dered</code> (<code>true/false</code> , overriding <code>--deredden</code> for that job). Cannot be combined with <code>--label</code> or <code>--phot-csv</code> , which apply to a single job.
<code>--phot-csv</code>	Fit this photometry file instead of the built one. For one-off tests.
<code>--dry-run</code>	Resolve and validate; write nothing.

`fit` needs all three of `--target`, `--recipe` and `--config` together, or `--jobs` instead. Otherwise it stops with `fit needs --target/--recipe/--config, or --jobs`.

7.5 batch and run

Flag	Meaning
<code>--roster</code>	<i>(batch only)</i> The roster JSON. Required. <code>run</code> writes its own and does not take this.
<code>--config</code>	The fit config applied to every galaxy. Required.
<code>--targets</code>	A CSV whose <code>name</code> column picks the galaxies. A <code>target</code> column works too.
<code>--target</code>	A single galaxy, instead of <code>--targets</code> . Passing both is an error.
<code>--recipe</code>	One recipe; omit for every applicable one.
<code>--label</code>	A suffix for every run directory. Defaults to the <code>name</code> in the fit config.
<code>--workers</code>	How many processes. Omit to choose automatically: up to 8 for eazy (one fewer than your CPUs), always 1 for Prospector. Never more than the number of jobs.
<code>--build</code>	<i>(batch only)</i> Rebuild each table before fitting it. <code>run</code> always builds and has no such flag.

Flag	Meaning
<code>--no-resume</code>	<i>(batch only)</i> Re-run galaxies already finished. <code>run</code> always resumes and has no such flag.
<code>--mw-ebv</code>	One E(B-V) for every galaxy instead of the per-galaxy lookup. On <code>batch</code> it needs both <code>--build</code> and <code>--deredden</code> ; on <code>run</code> , just <code>--deredden</code> , since <code>run</code> always builds.
<code>--report</code>	Where to write the report CSV. Defaults to <code>batch_report.csv</code> beside the roster.
<code>--log-dir</code>	Where to write the per-job logs (<code><target>__<recipe>.log</code>). Defaults to <code>batch_logs/</code> beside the report.
<code>--stop-after-failures</code>	Give up after this many failures. 0 disables the limit. Default 20.

`run` additionally takes `--catalog`, `--campaign` and `--out`, which behave exactly as they do for `roster`. It does **not** take `--roster`, `--build`, `--no-resume` or `--dry-run`.

A galaxy named in `--targets` that the roster does not declare is reported as **skipped** in the report rather than failing the batch — unless none of them is in the roster, which stops before any job runs.

8 What comes out

8.1 The directory tree

```
gal_0001/
  Photometry/
    gal_0001_catalog.csv    (yours, untouched)
    gal_0001_sed_plain.csv  the combined table
    gal_0001_sed_plain.provenance.json
  SED/
    plain/                  one folder per recipe
      eazy/                  one folder per backend
        64f456f8835f-quick_v1/ one run
          config.json
          phot.csv
          phot.provenance.json
          manifest.json
          summary.csv
          arrays.npz
          plots/
```

An eazy run writes a few more files than these — the filter and template records it used. A Prospector run writes `result.h5` instead of `summary.csv` and `arrays.npz`.

The long name on the run folder is not random. It is a fingerprint of the settings, the photometry and the filter curves. Fit the same galaxy the same way twice and you get the same folder — it is replaced, not duplicated. Change anything that matters and you get a new folder, so results from different settings can never be confused.

8.2 What each file is

File	What it holds
<code>_sed_<recipe>.csv</code>	The combined photometry the fit used
<code>.provenance.json</code>	Every input file, hashed, and every choice made
<code>config.json</code>	The complete settings, with defaults filled in
<code>phot.csv</code>	An exact copy of the photometry this run fitted
<code>manifest.json</code>	This run's result line
<code>summary.csv</code>	Redshifts and chi-square (eazy only)
<code>arrays.npz</code>	The redshift grid and the probability curve (eazy only)
<code>result.h5</code>	The chain, model stamps and run parameters (Prospector only)
<code>plots/</code>	The figures
<code>batch_report.csv</code>	One row per job — one galaxy under one recipe
<code>runs.jsonl</code>	Every run ever, appended

8.3 The numbers you probably want

`batch_report.csv` is the quickest place to look:

Column	Meaning
<code>target, recipe</code>	which galaxy, which recipe
<code>status</code>	ok, failed, or skipped
<code>stage</code>	which phase a job died in: <code>roster</code> , <code>applicable</code> , <code>build</code> , <code>plan</code> , <code>fit</code>
<code>zred_p50</code>	the redshift estimate (the median)
<code>zred_p16, zred_p84</code>	the 1-sigma range
<code>n_bands_fit</code>	how many bands actually entered the fit
<code>n_bands_table</code>	how many were available
<code>run_id, path</code>	where the run directory is
<code>seconds</code>	how long it took
<code>error</code>	why it failed, when it did

If `n_bands_fit` is much smaller than `n_bands_table`, bands were dropped — usually for low signal-to-noise. The run's `manifest.json` lists exactly which ones and why. When a job fails, read `stage` first: it says which phase gave out, which usually tells you whether the problem is in your files or in the fit.

The same numbers have two names

`summary.csv` calls the percentiles `z160`, `z500` and `z840`. The batch report and `manifest.json` call the identical numbers `zred_p16`, `zred_p50` and `zred_p84`. Searching a `summary.csv` for `zred_p50` finds nothing — look for `z500`.

9 Reading the figures

An eazy run writes two figures into `plots/` — three when `z_fixed` is set, which adds the SED at that fixed redshift. A Prospector run writes three different ones: a corner plot of the posterior, the sampler trace, and the SED at the best-fit parameters. The two described below are the eazy pair.

The **SED figure** shows your photometry as points, with error bars, against the best-fitting template. Points are colored by instrument. If the template misses a group of points badly, that instrument may be on the wrong flux scale — check the recipe.

The **redshift-scan figure** shows how well each redshift fits. A good fit shows one clear, narrow minimum. Two things to watch for:

- **Several comparable minima** — the fit cannot decide, and the quoted uncertainty understates the real one.
- **The minimum at the edge of the range** — the true redshift is probably outside your `z_min–z_max` window. The code warns about this in the log too. Widen the range and re-run.

10 When something goes wrong

Error messages name the file and say what to do. The common ones:

roster not found: ...

The path after `--roster` is wrong, or you have not run `sedfit roster` yet.

unknown target 'x'; the roster declares ...

A typo, or the galaxy was dropped during roster generation. Check `roster.json.report.csv` for why.

... has not been built; run: sedfit build ...

Exactly what it says — the message includes the command to run.

z_ref_kind 'reference' requires the roster to declare reference_redshift

A catalog row says `reference` but the campaign config has no `reference_redshift` for it to adopt. Add it to the campaign and regenerate the roster. Note this fires on the verb *after* `roster`, because generation does not check it.

sedfit: command not found

The environment exists but the package is not installed into it. From the repository folder: `conda activate <env>` then `pip install -e ..`. The `.yaml` files do not do this for you.

No module named 'pytest'

The base install does not include it. `pip install -e "[test]"`.

auto E(B-V) lookup needs astroquery ...

`--deredden` without `--mw-ebv` needs `astroquery`. Either `pip install -e "[dered]"` or supply the value yourself with `--mw-ebv`.

unknown bands: [...]

A band name in the campaign config is not one the package knows. The legal names are in `sedfit/data/registry.json`.

unknown provider token 'x' (known: [...])

A provider in the campaign config is not one of the ten legal tokens. The message lists them.

unknown recipe 'x'; the roster declares [...] or recipes not in the roster: [...]

The name after `--recipe` is not one the campaign defines. The message lists what is available.

templates=... is not a directory, a .param file, or a packaged set

The message lists the available set names. Check the spelling in your fit config.

only N fittable bands after inclusion and gates (min_valid_bands=5)

Too few bands survived. Either the photometry is thin for that galaxy, or the signal-to-noise cut removed too much. Lower `min_valid_bands` if that is scientifically appropriate — it sits

at the **top level** of the fit config, beside `err_floor`, not inside the `eazy` block. Its default is 5; `min_snr_broadband` defaults to 2.0 and `err_floor` to 0.05.

WARNING: 'gal_0001': chi2 minimum on the grid edge (z=0.6000); widen the grid
Increase `z_max` or decrease `z_min`. This is a warning, not an error: the run completes and writes results, but the redshift is pinned against the edge of the window you gave it and should not be trusted. Worth checking whenever you have not set the grid yourself — the defaults are 0.05 to 0.16.

existing run was produced by different machinery

You reinstalled or upgraded something since that run. Add `--force` to replace it.

no declared band in the roster report

The file exists but holds none of the bands the campaign config expects from it. Usually the `provider` or the band names are wrong.

A command seems to hang.

The first step of every verb opens every photometry file the roster declares. On a network drive or a cloud-synced folder, that can take minutes the first time. It is faster afterwards.

An unreadable error during a batch.

Re-run the failing galaxy on its own with `--workers 1`:

```
sedfit fit --roster roster.json --target gal_0007 \
  --recipe plain --config quick.json
```

Per-job logs from a batch are in `batch_logs/`, one file per galaxy-and-recipe, named `<target>__<recipe>.log`.

11 Getting more out of it

11.1 Several recipes at once

A recipe is one way of combining your photometry. Define several in the campaign config and `batch` runs them all, giving each its own folder. Comparing recipes is how you test whether a result depends on how the instruments were combined.

A recipe's **reference** says what defines the flux scale, and each source takes a **role**:

reference	What is truth
spherex	the SPHEREx spectrum; overlapping broadbands are rescaled onto it
anchors	one or two named broadband groups; the spectrum is moved onto them
none	nothing; every source stays at its measured level
role	What the source does
anchor	defines the reference. Legal only under anchors , which needs one or two. One rescales the spectrum without changing its color; two tilt it.
stitch	its whole instrument group is multiplied by one factor, measured in the reddest band it shares with the reference. Legal under spherex and anchors .
float	carried through untouched. Legal everywhere, and the only role legal under none .

A source can only stitch if it actually overlaps the reference; the build says so rather than guessing. `examples/README.md` works through four recipes covering every combination.

11.2 Comparing template sets

Copy your fit config, change `templates` and `name`, and run both. Because the run folder is named after the settings, the two results sit side by side without overwriting each other. The shipped sets are:

Set	What it is
<code>brown14_vac_cosmos160</code>	The recommended set: Brown+14 galaxies plus the COSMOS templates
<code>brown14</code>	The Brown+14 atlas as published
<code>brown14_vac</code>	Brown+14, air-to-vacuum corrected
<code>ssp_miles</code>	FSPS simple stellar populations, MILES library
<code>ssp_c3k_a</code>	The same on the C3K library

11.3 The central log

Every finished run appends one line to `runs.jsonl` under your data root. It is one JSON object per line, so it loads directly into pandas:

```
import pandas as pd
runs = pd.read_json("sed_fitting/runs.jsonl", lines=True)
```

That is the easiest way to compare many runs at once.

11.4 A worked example to copy

The repository ships a complete, runnable example in `examples/`: three synthetic galaxies with broadband photometry and SPHEREx spectra, the input files with every field annotated (including a Prospector config), and four recipes covering every way the package can combine instruments. Run it as-is:

```
cd examples
sedfit run --catalog sample_catalog.csv --campaign campaign.json \
    --config fit_quick.json
```

Then copy `sample_catalog.csv`, `campaign.json` and `fit_quick.json` into your own working directory and replace their contents. `examples/README.md` explains every field.

11.5 Where to read next

`docs/technical_manual.md` in the repository explains how the code works internally — what each module does, how the flux scaling is computed, and what every output field means.